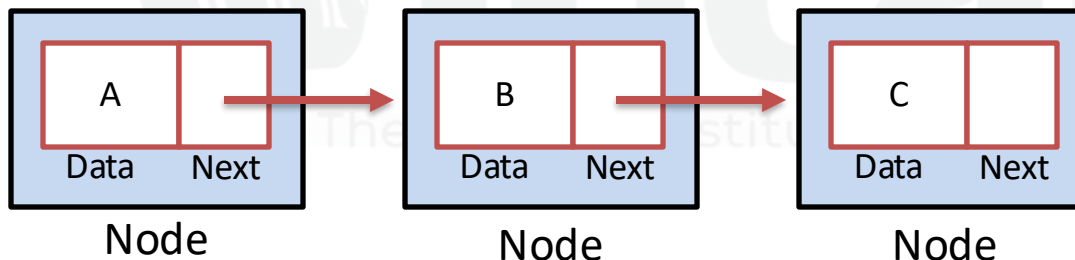


Linked List

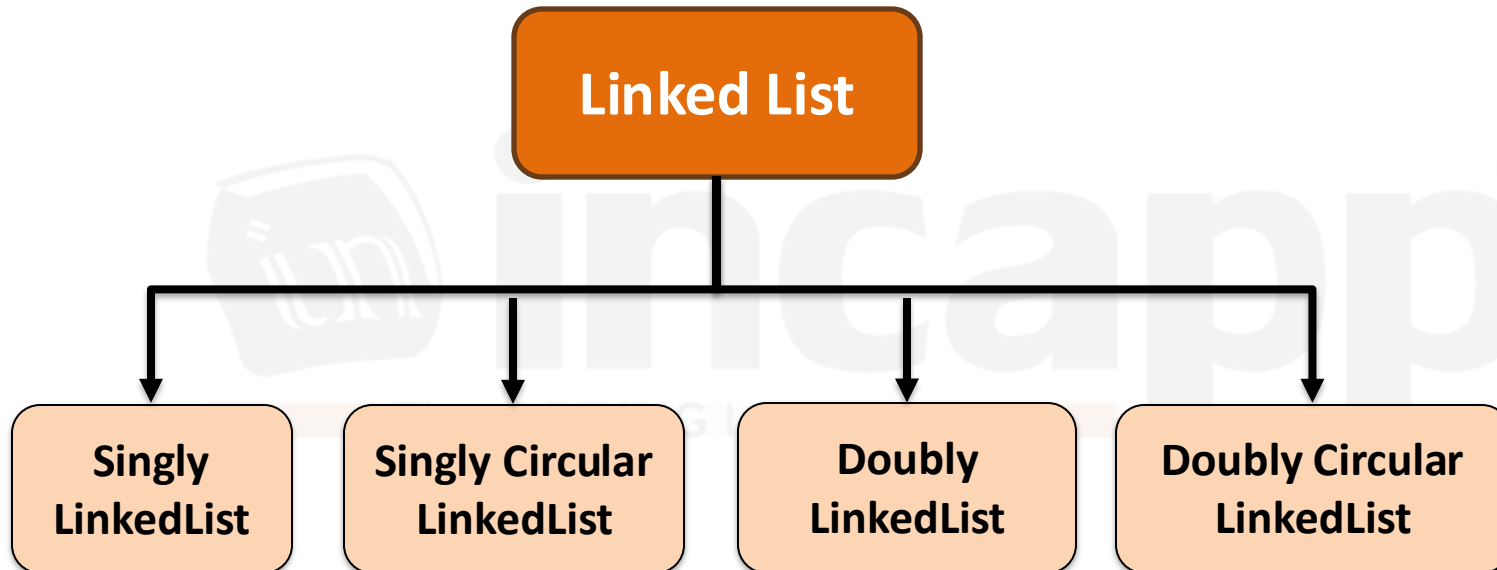
- **Linked List** is a data structure that stores elements in a non-contiguous location. It is a **linear data structure**.
- Each data item is called a '**Node**' and each node has a data part and an reference part. The reference part stores the link to the next node in the LinkedList.



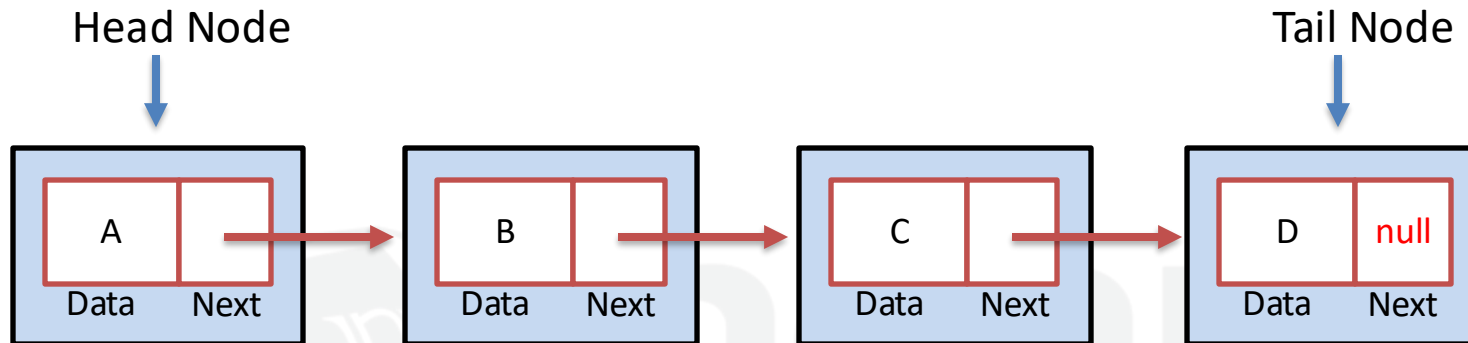
Why LinkedList is a Dynamic Data Structure?

LinkedList size is not fixed. Size of LinkedList can be increase or decrease, without creating new LinkedList.

Types of Linked List



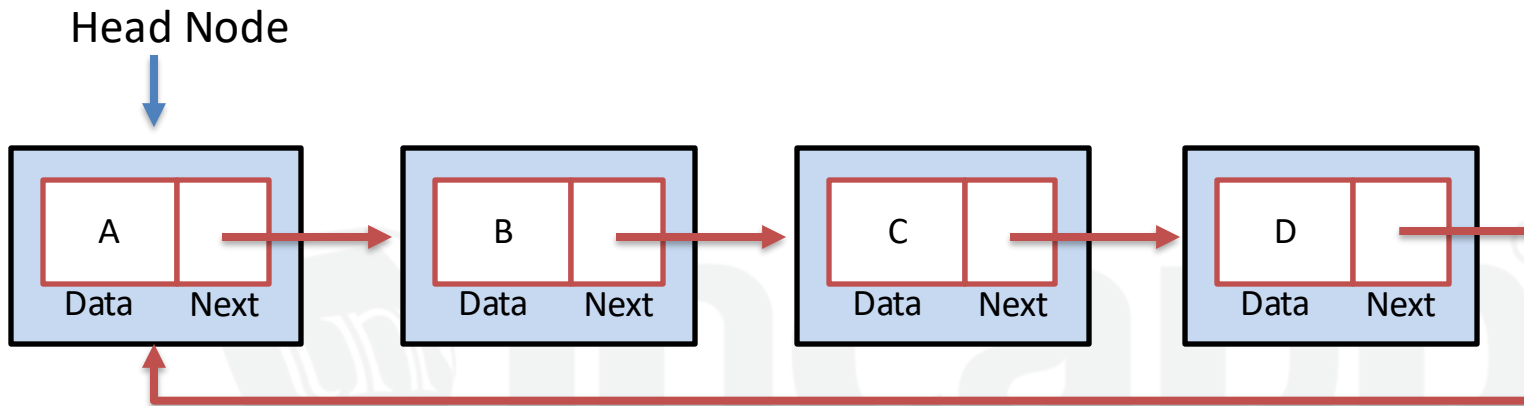
Singly Linked List



The **Head** of the LinkedList contains the reference/address of the first element in the LinkedList. The last node in the LinkedList is the **Tail**.

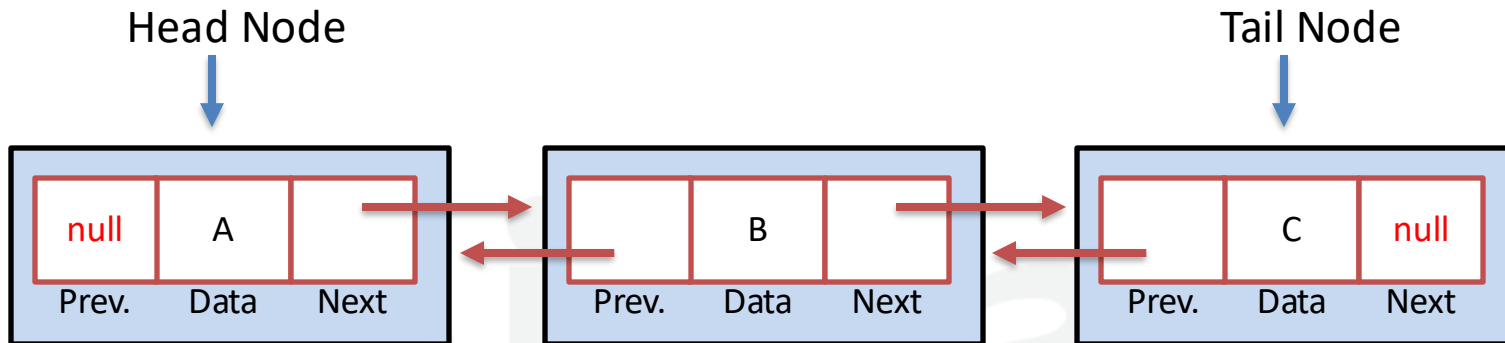
Singly-linked List stores the reference of only the next node in the LinkedList. The Last node does not refer any node that means last node contains “null” as next value.

Singly Circular Linked List



In the **Singly Circular Linked List**, the last node in the LinkedList contains the reference of the head node in the LinkedList.

Doubly Linked List

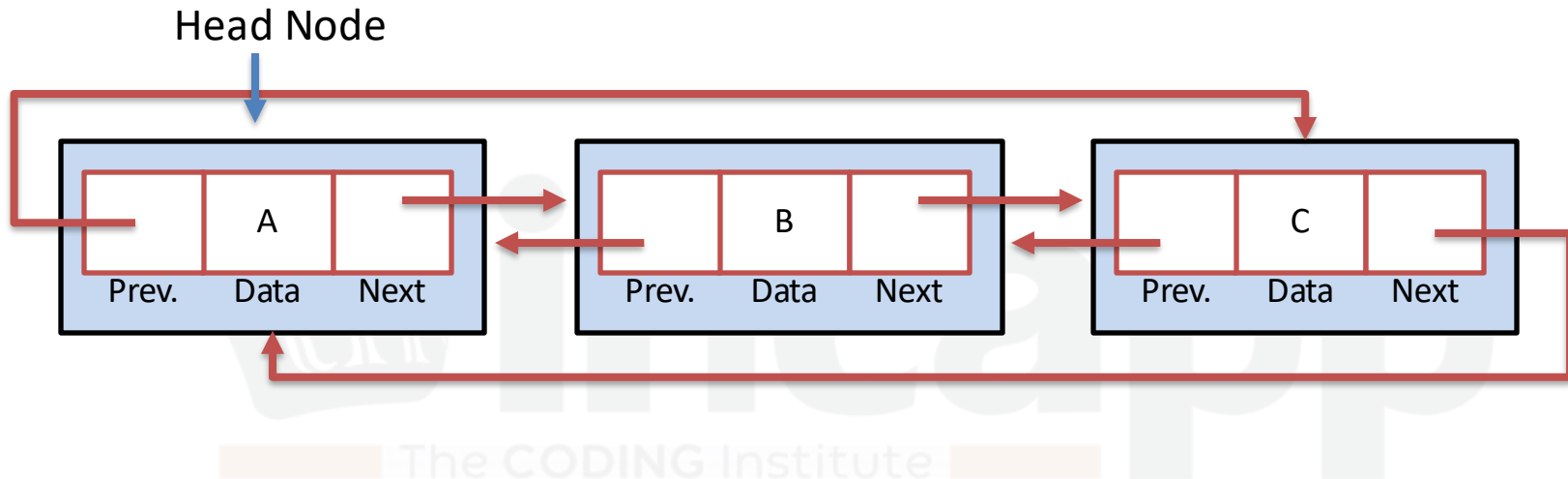


In **Doubly Linked List**, each node has three parts:

1. Reference to the previous node in the LinkedList.
2. Data part
3. Reference to the next node in the LinkedList.

The previous value of the first/head node in the LinkedList will be set to “null” and the next value of the last/tail node in the LinkedList is set to “null”.

Doubly Circular Linked List



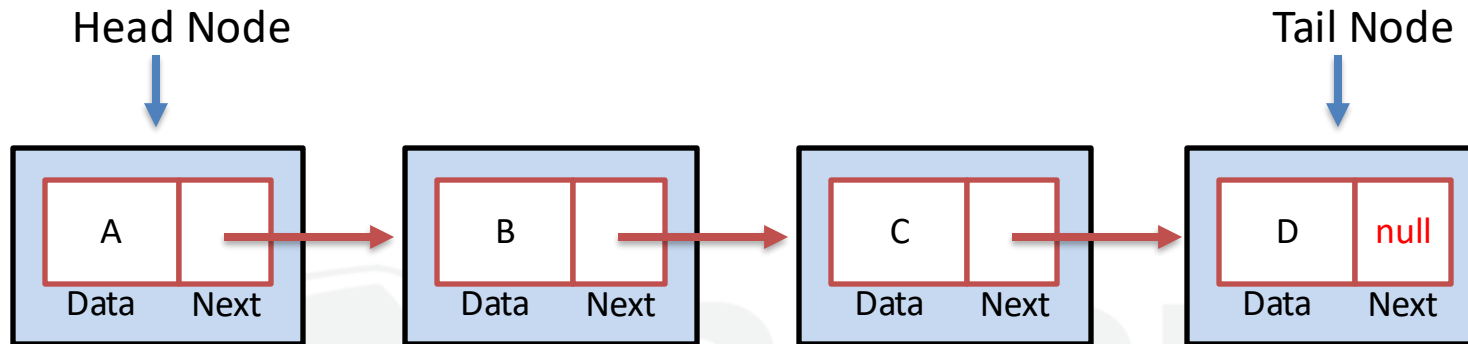
The **Circular Doubly Linked List** is a combination of the Doubly Linked List and the Circular Linked List. It means that the last node in the Doubly Linked List contains the reference of the first/head node and first node contains the reference of last node.

Singly Linked List



Singly Linked List

Singly Linked List



The **Head** of the LinkedList contains the reference/address of the first element in the LinkedList. The last node in the LinkedList is the **Tail**.

Singly-linked List stores the reference of only the next node in the LinkedList. The Last node does not refer any node that means last node contains “null” as next value.

Singly Linked List - Operations



Algo for Creating Linked List or Insertion at End of Linked List

Let's define a **Node** with two members **data** and **next**. Define two node variables '**head**' and '**tail**' and set them to **null**.

1. Create a **newNode** with given value.
2. Check whether list is Empty, if (**head == null**) then,
 - a) Set **head = newNode**
 - b) Set **tail = newNode**
3. Otherwise,
 - a) Set **tail.next = newNode**
 - b) Set **tail = newNode**
4. Exit.

Singly Linked List - Operations

Algo for Traversing Linked List

Let '**head**', '**tail**' and '**currentNode**' are the node variables.

1. Check, if(**head**==null)
 - a) Print, LinkedList is Empty
2. Otherwise,
 - a) Set **currentNode** = head
 - b) Repeat, while (**currentNode** != null)
 - i. Print, **currentNode.data**
 - ii. Set **currentNode** = **currentNode.next**
3. Exit.

Singly Linked List - Operations



Algo for Insertion at Beginning of Linked List

Let '**head**' and '**tail**' are the node variables.

1. Create a **newNode** with given value.
2. Check, if(**head==null**)
 - a) Set **head = newNode**
 - b) Set **tail = newNode**
3. Otherwise,
 - a) Set **newNode.next = head**
 - b) Set **head = newNode**
4. Exit.

Singly Linked List - Operations

Algo for Insertion after Specific Value of Linked List

Let '**head**', '**tail**' and '**currentNode**' are the node variables. **locationValue** is the node value after which we want to insert the **newNode**.

1. Read **locationValue**
2. Set **currentNode = head**
3. Set **flag = true**
4. Traverse each node to find the location value.
Repeat, while (**currentNode != null**)
 - a) Check, if (**currentNode.data == locationValue**) then:
 - i. Set **flag = false**
 - ii. Break the loop
 - b) Set **currnetNode = currentNode.next**

continue...

Singly Linked List - Operations



5. Check, if (**flag == true**) then:
 - a) Print, Location Value does not exist.
6. Otherwise,
 - a) Create a **newNode** with given value.
 - b) Set **newNode.next = currentNode.next**
 - c) Set **currentNode.next = newNode**
 - d) Check, if(**currentNode == tail**)
 - Set **tail= newNode**
7. Exit.

Singly Linked List - Operations

Algo for Deletion from End of Linked List

Let 'head', 'tail' and 'currentNode' are the node variables.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head==tail**) then:
 - a) Set **head = tail = null**
3. Otherwise,
 - a) Set **currentNode = head**
 - b) Repeat, while (**currentNode.next != tail**)
 - Set **currentNode = currentNode.next**
 - c) Set **currentNode.next = null**
 - d) Set **tail = currentNode**
4. Exit.

Singly Linked List - Operations



Algo for Deletion from Beginning of Linked List

Let '**head**' and '**tail**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head==tail**) then:
 - a) Set **head = tail = null**
3. Otherwise,
 - a) Set **head = head.next**
4. Exit.

Singly Linked List - Operations

Algo for Deletion of Specific Value of Linked List

Let **locationValue** is the node value which we want to delete.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.data == locationValue**) then:
 - a) Set **head = head.next**
3. Otherwise,
 - a) Set **currentNode = head.next**
 - b) Set **previousNode = head**
 - c) Set **flag = true**
 - d) Repeat, while (**currentNode != null**)
 - i. Check, if(**currentNode.data == locationValue**) then
 1. Set **flag = false**
 2. Break the loop
 - ii. Set **previousNode = currentNode**
 - iii. Set **currentNode = currentNode.next**

Continue...

Singly Linked List - Operations



- e) Check, if(**flag == true**) then
 - Print, Value does not exist.
- f) Otherwise,
 - Set **previousNode.next = currentNode.next**
 - Check, if(**currentNode == tail**)
 - Set **tail = previousNode**

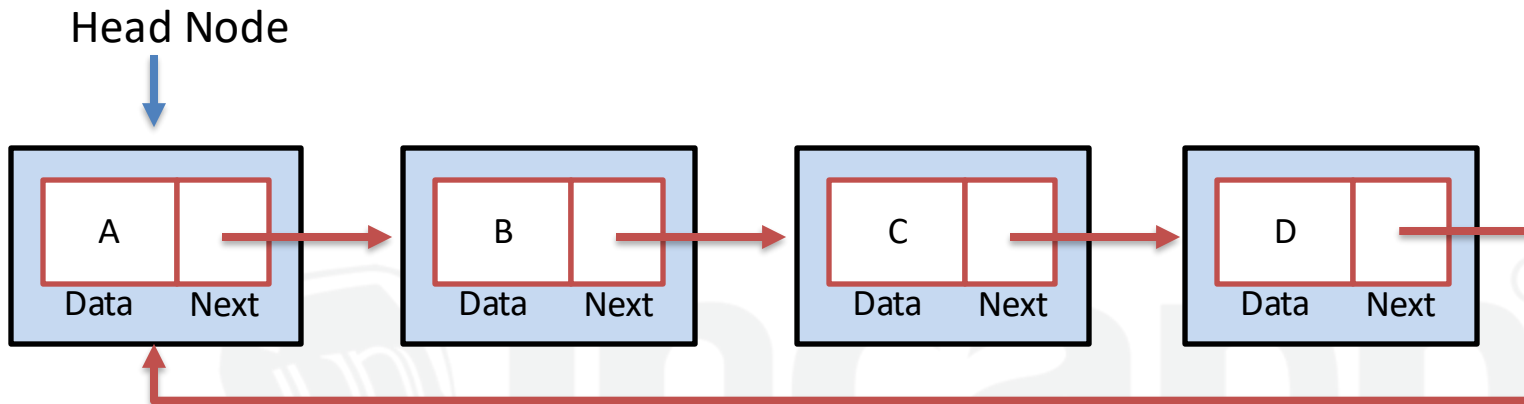
4. Exit.

Singly Circular Linked List



Singly Circular Linked List

Singly Circular Linked List



In the **Singly Circular Linked List**, the last node in the LinkedList contains the reference of the head node in the LinkedList.

Singly Circular Linked List- Operations

Algo for Creating Circular Linked List or Insertion at End of Linked List

Let's define a **Node** with two members **data** and **next**. Define one node variable '**head**' and set it to **null**.

1. Create a **newNode** with given value.
2. Check whether list is Empty, if (**head == null**) then,
 - a) Set **head = newNode**
 - b) Set **newNode.next = head**
3. Otherwise,
 - a) Set **currentNode = head**
 - b) Repeat, while (**currentNode.next != head**)
 - **currentNode = currentNode.next**
 - c) Set **currentNode.next = newNode**
 - d) Set **newNode.next = head**
4. Exit.

Singly Circular Linked List- Operations

Algo for Traversing Linked List

Let '**head**' and '**currentNode**' are the node variables.

1. Check, if(**head**==null)
 - a) Print, LinkedList is Empty
2. Otherwise,
 - a) Set **currentNode** = head
 - b) Repeat, do
 - i. Print, **currentNode.data**
 - ii. Set **currentNode** = **currentNode.next**
3. Exit.

Singly Circular Linked List- Operations

Algo for Insertion at Beginning of Linked List

Let '**head**' and '**currentNode**' are the node variables.

1. Create a **newNode** with given value.
2. Check, if(**head==null**)
 - a) Set **head = newNode**
 - b) Set **newNode.next = head**
3. Otherwise,
 - a) Set **currentNode = head**
 - b) Repeat, while (**currentNode.next != head**)
 - **currentNode = currentNode.next**
 - c) Set **currentNode.next = newNode**
 - d) Set **newNode.next = head**
 - e) Set **head = newNode**
4. Exit.

Singly Circular Linked List- Operations

Algo for Insertion after Specific Value of Linked List

Let '**head**' and '**currentNode**' are the node variables. **locationValue** is the node value after which we want to insert the **newNode**.

1. Read **locationValue**
 2. Set **currentNode = head**
 3. Set **flag = true**
 4. Traverse each node to find the location value.
Repeat, do
 - a) Check, if (**currentNode.data == locationValue**) then:
 - i. Set **flag = false**
 - ii. Break the loop
 - b) Set **currnetNode = currentNode.next**
- while (**currentNode != head**)

continue...

Singly Circular Linked List- Operations



5. Check, if (**flag == true**) then:
 - a) Print, Location Value does not exist.
6. Otherwise,
 - a) Create a **newNode** with given value.
 - b) Set **newNode.next = currentNode.next**
 - c) Set **currentNode.next = newNode**
7. Exit.

Singly Circular Linked List- Operations

Algo for Deletion from End of Linked List

Let '**head**' and '**currentNode**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.next == head**) then:
 - a) Set **head = null**
3. Otherwise,
 - a) Set **currentNode = head.next**
 - b) Set **previousNode = head**
 - c) Repeat, while (**currentNode.next != head**)
 - i. Set **previousNode = currentNode**
 - ii. Set **currentNode = currentNode.next**
 - d) Set **previousNode.next = head**
4. Exit.

Singly Circular Linked List- Operations

Algo for Deletion from Beginning of Linked List

Let '**head**' and '**currentNode**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.next == head**) then:
 - a) Set **head = null**
3. Otherwise,
 - a) Set **currentNode = head.next**
 - b) Repeat, while (**currentNode.next != head**)
 - Set **currentNode = currentNode.next**
 - c) Set **head = head.next**
 - d) Set **currentNode.next = head**
4. Exit.

Singly Circular Linked List- Operations

Algo for Deletion of Specific Value of Linked List

Let **locationValue** is the node value which we want to delete.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.data == locationValue** and **head.next == head**) then
 - a) Set **head = null**
3. Otherwise, if (**head.data == locationValue** and **head.next != head**) then
 - a) Set **currentNode = head**
 - b) Repeat, while (**currentNode.next != head**)
 - Set **currentNode = currentNode.next**
 - c) Set **head = head.next**
 - d) Set **currentNode.next = head**
4. Otherwise,
 - a) Set **currentNode = head**
 - b) Set **previousNode = null**
 - c) Set **flag = true**

Continue...

Singly Circular Linked List- Operations



- d) Repeat, do
 - i. Check, if(**currentNode.data == locationValue**) then
 - i. Set **flag = false**
 - ii. Break the loop
 - ii. Set **previousNode = currentNode**
 - iii. Set **currentNode = currentNode.next**
- while (**currentNode != head**)
- e) Check, if(**flag == true**) then
 - Print, Value does not exist.
- f) Otherwise,
 - Set **previousNode.next = currentNode.next**

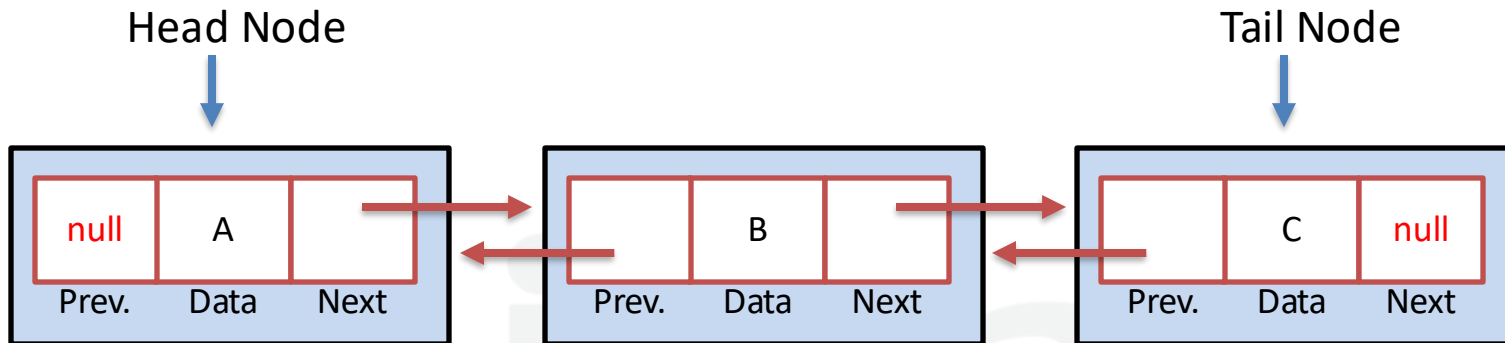
4. Exit.

Doubly Linked List



Doubly Linked List

Doubly Linked List



In **Doubly Linked List**, each node has three parts:

1. Reference to the previous node in the LinkedList.
2. Data part
3. Reference to the next node in the LinkedList.

The previous value of the first/head node in the LinkedList will be set to “null” and the next value of the last/tail node in the LinkedList is set to “null”.

Doubly Linked List - Operations

Algo for Creating Linked List or Insertion at End of Linked List

Let's define a **Node** with three members **previous**, **data** and **next**. Define two node variables '**head**' and '**tail**' and set them to **null**.

1. Create a **newNode** with given value.
2. Check whether list is Empty, if (**head == null**) then,
 - a) Set **head = newNode**
 - b) Set **tail = newNode**
3. Otherwise,
 - a) Set **tail.next = newNode**
 - b) Set **newNode.previous = tail**
 - c) Set **tail = newNode**
4. Exit.

Doubly Linked List - Operations

Algo for Forward Traversing Linked List

Let '**head**', '**tail**' and '**currentNode**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, LinkedList is Empty
2. Otherwise,
 - a) Set **currentNode = head**
 - b) Repeat while (**currentNode != null**)
 - i. Print **currentNode.data**
 - ii. Set **currentNode = currentNode.next**
3. Exit.

Doubly Linked List - Operations

Algo for Backward Traversing Linked List

Let 'head', 'tail' and 'currentNode' are the node variables.

1. Check, if(**head==null**)
 - a) Print, LinkedList is Empty
2. Otherwise,
 - a) Set **currentNode = tail**
 - b) Repeat while (**currentNode != null**)
 - i. Print **currentNode.data**
 - ii. Set **currentNode = currentNode.previous**
3. Exit.

Doubly Linked List - Operations

Algo for Insertion at Beginning of Linked List

Let '**head**' and '**tail**' are the node variables.

1. Create a **newNode** with given value.
2. Check, if(**head==null**)
 - a) Set **head = newNode**
 - b) Set **tail = newNode**
3. Otherwise,
 - a) Set **head.previous = newNode**
 - b) Set **newNode.next = head**
 - c) Set **head = newNode**
4. Exit.

Doubly Linked List - Operations

Algo for Insertion after Specific Value of Linked List

Let '**head**', '**tail**' and '**currentNode**' are the node variables. **LocationValue** is the node value after which we want to insert the **newNode**.

1. Read **locationValue**
2. Check, if(**locationValue == tail.data**)
 - a) Create a **newNode** with given value.
 - b) Set **newNode.previous = tail**
 - c) Set **tail.next = newNode**
 - d) Set **tail = newNode**
3. Otherwise,
 - a) Set **currentNode = head**
 - b) Set **flag = true**

Continue...

Doubly Linked List - Operations



- c) Traverse each node to find the location value.
Repeat, while (**currentNode != null**)
- i. Check, if (**currentNode.data == locationValue**) then:
 - Set **flag = true**
 - Break the loop
 - ii. Set **currnetNode = currentNode.next**
- d) Check if (**flag == true**) then:
- Print, Location Value does not exist.
- e) Otherwise,
- Create a **newNode** with given value
 - Set **currnetNode.next.previous = newNode**
 - Set **newNode.next = currnetNode.next**
 - Set **currnetNode.next = newNode**
 - Set **newNode.previous = currnetNode**
6. Exit.

Doubly Linked List - Operations

Algo for Deletion from End of Linked List

Let '**head**' and '**tail**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head==tail**) then:
 - a) Set **head = tail = null**
3. Otherwise,
 - a) Set **tail = tail.previous**
 - b) Set **tail.next = null**
4. Exit.

Doubly Linked List - Operations

Algo for Deletion from Beginning of Linked List

Let 'head' and 'tail' are the node variables.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head==tail**) then:
 - a) Set **head = tail = null**
3. Otherwise,
 - a) Set **head = head.next**
 - b) Set **head.previous = null**
4. Exit.

Doubly Linked List - Operations

Algo for Deletion of Specific Value of Linked List

Let **locationValue** is the node value which we want to delete.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.data == locationValue** and **head == tail**) then:
 - a) Set **head = tail = null**
3. Otherwise
 - a) Set **currentNode = head**
 - b) Traverse each node to find the location value.
Repeat, while (**currentNode != null**)
 - i. Check, if(**currentNode.data == locationValue**) then
 1. Set **flag = false**
 2. Break the loop
 - ii. **currentNode = currentNode.next**
 - d) Check, if(**flag == true**) then
 - Print, Value does not exist.

Continue...

Doubly Linked List - Operations



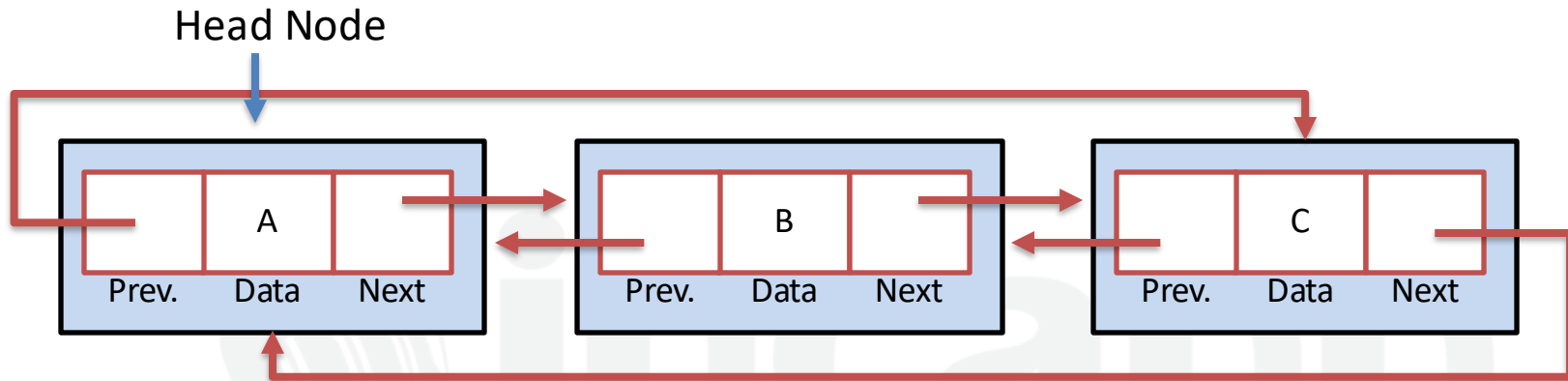
- e) Otherwise
 - i. Check, if(**currentNode** == **head**) then
 - 1. Set **head** = **head.next**
 - 2. Set **head.previous** = **null**
 - ii. Otherwise, if(**currentNode** == **tail**) then
 - 1. Set **tail** = **tail.previous**
 - 2. Set **tail.next** = **null**
 - iii. Otherwise, then
 - 1. Set **currentNode.next.previous** = **currentNode.previous**
 - 2. Set **currentNode.previous.next** = **currentNode.next**
- 4. Exit.

Doubly Circular Linked List



Doubly Circular Linked List

Doubly Circular Linked List



The **Circular Doubly Linked List** is a combination of the Doubly Linked List and the Circular Linked List. It means that the last node in the Doubly Linked List contains the reference of the first/head node and first node contains the reference of last node.

Doubly Circular Linked List - Operations

Algo for Creating Linked List or Insertion at End of Linked List

Let's define a **Node** with three members **previous**, **data** and **next**. Define a node variable '**head**' and set it to **null**.

1. Create a **newNode** with given value.
2. Check whether list is Empty, if (**head == null**) then,
 - a) Set **head = newNode**
 - b) Set **head.next = head**
 - c) Set **head.prev = head**
3. Otherwise,
 - a) Set **newNode.next = head**
 - b) Set **newNode.previous = head.previous**
 - c) Set **head.previous.next = newNode**
 - d) Set **head.previous = newNode**
4. Exit.

Doubly Circular Linked List - Operations

Algo for Forward Traversing Linked List

Let '**head**' and '**currentNode**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, LinkedList is Empty
2. Otherwise,
 - a) Set **currentNode = head**
 - b) Repeat, do
 - i. Print, **currentNode.data**
 - ii. Set **currentNode = currentNode.next**
 - c) while (**currentNode != head**)
3. Exit.

Doubly Circular Linked List - Operations

Algo for Backward Traversing Linked List

Let '**head**' and '**currentNode**' are the node variables.

1. Check, if(**head==null**)
 - a) Print, LinkedList is Empty
2. Otherwise,
 - a) Set **currentNode = head.previous**
 - b) Repeat, do
 - i. Print, **currentNode.data**
 - ii. Set **currentNode = currentNode.previous**
 - c) while (**currentNode != head.previous**)
3. Exit.

Doubly Circular Linked List - Operations

Algo for Insertion at Beginning of Linked List

Let '**head**' are the node variables.

1. Create a **newNode** with given value.
2. Check, if(**head==null**)
 - a) Set **head = newNode**
 - b) Set **head.next = head**
 - c) Set **head.previous = head**
3. Otherwise,
 - a) Set **newNode.next = head**
 - b) Set **newNode.previous = head.previous**
 - c) Set **head.previous.next = newNode**
 - d) Set **head.previous = newNode**
 - e) Set **head = newNode**
4. Exit.

Doubly Circular Linked List - Operations

Algo for Insertion after Specific Value of Linked List

Let '**head**' and '**currentNode**' are the node variables. **LocationValue** is the node value after which we want to insert the **newNode**.

1. Read **locationValue**
2. Set **currentNode = head**
3. Set **flag = true**
4. Repeat, do
 - a) Otherwise, if (**currentNode.data == locationValue**) then:
 - Set **flag = false**
 - Break the loop
 - b) Set **currentNode = currentNode.next**
while (**currentNode != head**)
5. Check, if (**flag == true**) then:
 - a) Print, Location Value does not exist.

continue...

Doubly Circular Linked List - Operations



6. Otherwise,
 - a) Create a **newNode** with given value.
 - b) Set **newNode.next = currentNode.next**
 - c) Set **newNode.previous = currentNode**
 - d) Set **currentNode.next.previous = newNode**
 - e) Set **currentNode.next = newNode**
7. Exit.

Doubly Circular Linked List - Operations

Algo for Deletion from End of Linked List

Let 'head' is the node variable.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.next==head**) then:
 - a) Set **head = null**
3. Otherwise,
 - a) Set **head.previous.previous.next = head**
 - b) Set **head.previous = head.previous.previous**
4. Exit.

Doubly Circular Linked List - Operations

Algo for Deletion from Beginning of Linked List

Let '**head**' is the node variable.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.next==head**) then:
 - a) Set **head = null**
3. Otherwise,
 - a) Set **head.next.previous = head.previous**
 - b) Set **head.previous.next = head.next**
 - c) Set **head = head.next**
4. Exit.

Doubly Circular Linked List - Operations

Algo for Deletion of Specific Value of Linked List

Let **locationValue** is the node value which we want to delete.

1. Check, if(**head==null**)
 - a) Print, List is Empty, Nothing to delete.
2. Otherwise, if (**head.data == locationValue** and **head.next == head**) then:
 - a) Set **head = null**
3. Otherwise, if (**head.data == locationValue**) then:
 - a) Set **head.previous.next = head.next**
 - b) Set **head.next.previous = head.previous**
 - c) Set **head = head.next**
4. Otherwise,
 - a) Set **currentNode = head**
 - b) Set **flag = true**

continue...

Doubly Circular Linked List - Operations

c) Traverse each node to find the location value.

Repeat, do

i. Check, if(**currentNode.data == locationValue**) then

- Set **flag = false**
- Break the loop

ii. **currentNode = currentNode.next**

while (**currentNode != head**)

d) Check, if(**flag == true**) then

- Print, Value does not exist.

e) Otherwise,

- Set **currentNode.next.previous = currentNode.previous**
- Set **currentNode.previous.next = currentNode.next**

6. Exit.